ITAI-1371 - Intro to Machine Learning

Reflective Journal

Camila Ferreira da Silva

While fixing and running the Lab 8 notebook, I gained a better understanding of the bias-variance tradeoff. Comparing the polynomial models showed me that degree 1 was too simple to capture the sine wave, while degree 15 was complex enough to follow the noise. Degree 4 provided a better balance by capturing the overall pattern. The learning curves also helped me understand how training and cross-validation scores can reveal underfitting and overfitting. This lab showed me that a model's training performance alone does not explain everything, it is also important to consider how well the model generalizes. In future projects, I would like to explore Ridge and Lasso regression to learn how regularization can improve generalization.

Danh Nguyen

My part was building the degree 15 model and explaining why it overfits. Before this lab, I thought a lower error always meant a better model. Degree 15 changed that: it had the lowest training error (about 0.21) but the highest cross-validation error (about 1.03). That gap showed me what high variance really means. The model memorized the noise instead of learning the sine-wave pattern, like a student who memorizes answers but fails the exam. I also learned that a plot alone can hide overfitting, since the curve still looked roughly reasonable. Only testing on unseen data proved the problem, which helped me understand why the train/test split from earlier labs matters.

Great Inioluwa Olayiwola

1. Conceptual Breakthroughs: Deconstructing Complexity

The hands-on evaluation of polynomial degrees (1, 4, and 15) provided a clear visual illustration of the Goldilocks principle in predictive modeling. Initially, I assumed higher-degree models were inherently superior due to increased mathematical capacity. However, the degree 1 model demonstrated extreme simplicity by fitting a rigid line across a curved sine wave (high bias/underfitting). Conversely, the degree 15 model overfitted significantly, rather than extracting the structural trend, it aggressively chased individual noisy data points (high variance). The degree 4 polynomial struck the perfect balance, capturing the true wave pattern while remaining resilient to noise.

2. Diagnostic Tools: Decoding Learning Curves

Visualizing training vs cross-validation scores illuminated key diagnostic signals for model performance. At the first degree of underfitting, adding more data is useless for improving the score. For a high degree of overfitting (like degree 15) the scores also remain low with more data, but this time it is because the model is learning to memorize instead of generalize. This can be seen from the big gap between training and cross-validation scores.

3. Reflection on Strategy & Evolving Perspective

This lab changed my problem-solving approach. Rather than relying solely on raw training accuracy, I now evaluate the convergence gap between validation metrics to diagnose model health. Moving forward, I plan to explore regularization techniques such as Ridge and Lasso regression to mathematically constrain feature coefficients, allowing complex models to generalize effectively without manual degree reduction.

Karima Sadruddin

In this lab, I learned about the bias-variance tradeoff and how it affects the performance of machine learning models. I explored three different polynomial regression models: Degree 1, Degree 4, and Degree 15. By comparing their graphs, I understood the difference between underfitting, overfitting, and a good fit.

One important thing I learned was that a model that is too simple may not capture the actual patterns in the data. This is called underfitting and is represented by the Degree 1 model. On the other hand, a model that is too complex may learn the training data too closely, including random noise. This is called overfitting and is represented by the Degree 15 model.

I found the Degree 4 model particularly interesting because it captured the general pattern of the data without making the curve unnecessarily complicated. It provided a good balance between underfitting and overfitting. Creating and plotting this model also helped me understand how polynomial regression works in Python.

I also learned about learning curves and how they help us compare training scores with cross-validation scores. These curves are useful for identifying whether a model has high bias or high variance and whether it can perform well on new data.

Overall, this lab helped me understand that building a successful machine learning model is not just about getting a high training score. It is important to find the right level of complexity so that the model can learn meaningful patterns and make reliable predictions on unseen data. This knowledge will be useful in my future machine learning projects because choosing the right model is an important part of achieving accurate results.

Quan Thach

Before this lab, I understood overfitting and underfitting mostly as definitions. Seeing them on actual plots changed that. In the first task, the degree 1 line missed the curve of the data entirely, while the degree 15 curve wiggled to chase every noisy point. The degree 4 model sat in between and followed the real pattern without reacting to the noise. That made the bias-variance tradeoff feel concrete rather than abstract.

The learning curve section added another layer. Looking at the degree 1 curve, I learned that a low training score is the key warning sign of underfitting. When both the training and validation scores stay low and close together, the problem is the model's simplicity, not the amount of data. That was a useful shift in thinking, because my first instinct when a model performs poorly is often to collect more data, and here that would not have helped.

I also ran into a practical issue: the learning curve function in the notebook had lost its line breaks, so it couldn't run. Fixing it reminded me to read code carefully before trusting it. I

also noticed that because the data was sorted, the default cross-validation could give odd validation scores, and that shuffling the folds is a sensible fix. Going forward, I plan to check how my data is ordered before choosing a validation strategy, and to use learning curves as a regular diagnostic tool to decide whether a model needs more complexity or more data.